

GUIA 2

Arquitetura e Tecnologia de Computadores II

Catarina Sousa A97259

Francisca Domingues A104953

Descrição textual do algoritmo

O programa inicia com a configuração dos periféricos necessários: UART0 para a comunicação série, o Timer0 como base temporal do PCA, e o PCA0 em modo PWM para gerar a saída modulada. O duty-cycle inicial foi definido como 50%, ficando guardado numa variável permanente na memória.

Durante a execução, o programa verifica continuamente se existem caracteres recebidos pela UART0. Cada carácter recebido é entregue à máquina de estados do protocolo. A receção da trama passa pelos estados de espera, pelo carácter inicial T, pela leitura dos dois dígitos do comprimento cc, pela leitura dos restantes caracteres da trama e pela validação final.

Na versão final da Fase 2, apenas são aceites tramas com o seguinte formato:

*Tcc<dados>*hh\n*

Depois de recebida a trama completa, o programa valida o carácter inicial, o campo de comprimento, o terminador de linha, a posição do separador * e o checksum.

O checksum recebido em ASCII hexadecimal é convertido para um valor numérico e comparado com o checksum calculado localmente. Se a trama for inválida, é enviada a resposta ERR. Se for válida, o campo <dados> é interpretado como comando.

Os comandos implementados permitem definir diretamente o duty-cycle, incrementá-lo ou decrementá-lo, consultar o seu estado atual, ligar ou desligar a saída PWM, alterar a frequência PWM e controlar a sua reprodução numa sequência de pontos.

Comando	Função
D0 ... D100	Define diretamente o duty-cycle entre 0% e 100%
D+	Incrementa o duty-cycle em 1%, sem ultrapassar 100%
D-	Decrementa o duty-cycle em 1%, sem descer abaixo de 0%
D?	Solicita o estado atual da saída e o duty-cycle armazenado
START	Ativa a saída PWM usando o duty-cycle guardado
STOP	Desativa a saída PWM sem parar o contador do PCA através do bit CR
Fxxx	Define a frequência PWM dentro da gama implementada, entre 100 Hz e 1000 Hz
S:nn	Inicia o carregamento de uma sequência com nn pontos
SON	Inicia a reprodução da sequência carregada
SOFF	Termina a reprodução da sequência

No comando START, o canal PWM do PCA é ativado e o valor de duty-cycle armazenado é aplicado à saída. No comando STOP, a saída é desligada desativando o canal PWM e forçando o pino ao estado OFF, sem parar o PCA através do bit PCA0CN0_CR.

No modo de sequência, após o comando S:nn, o programa passa a receber valores ASCII, um por linha, validando que cada ponto está entre 0 e 100. Quando todos os pontos são recebidos, a sequência pode ser reproduzida com SON. Durante a reprodução, o primeiro ponto é aplicado imediatamente quando é recebido esse comando. Os pontos seguintes são aplicados periodicamente através da interrupção do módulo 1 do PCA, usando um contador auxiliar para tornar a variação do brilho visível. A reprodução da sequência é interrompida com SOFF.

Variáveis em memória, tipos e periféricos utilizados

Variáveis principais

Variável	Tipo	Função
duty_percent	static unsigned char	Guarda o duty-cycle atual, entre 0 e 100%. Valor inicial: 50%
pwm_enabled	static bit	Indica se a saída PWM está ativa ou inativa
pwm_freq	static unsigned int	Guarda a frequência PWM configurada (100 Hz a 1000 Hz)
sequence_data[]	static unsigned char xdata	Guarda os pontos da sequência de duty-cycle
sequence_len	static volatile unsigned char	Número de pontos carregados na sequência
sequence_index	static volatile unsigned char	Índice do ponto atualmente reproduzido
seq_playing	static volatile bit	Indica se a sequência está em reprodução
frame[]	static char	Buffer usado para guardar a trama recebida
frame_len	static unsigned char	Número de caracteres já recebidos da trama
frame_state	static frame_state_t	Estado atual da máquina de estados da recepção da trama
protocol_state	static protocol_state_t	Indica se o programa está a receber tramas normais ou pontos da sequência
seq_line[]	static char	Buffer temporário para receber cada ponto da sequência
seq_expected	static unsigned char	Número de pontos esperados após o comando S:nn
seq_received	static unsigned char	Número de pontos já recebidos
seq_delay	static volatile unsigned char	Contador auxiliar usado para atrasar a mudança entre pontos da sequência, tornando a variação do brilho visível

Estruturas e tipos utilizados

Estrutura	Categoria	Descrição
fifo_t	Estrutura	Implementa uma fila circular, contendo o apontador para o buffer, os índices de leitura/escrita e a flag de estado da FIFO.
command_result_t	Enumeração	Indica o resultado da execução de um comando, podendo representar erro, sucesso ou entrada em modo de carregamento da sequência.
protocol_state_t	Enumeração	Distingue o modo normal de receção de tramas do modo de receção dos pontos da sequência.
frame_state_t	Enumeração	Representa os estados da máquina de receção da trama durante o processamento dos caracteres recebidos pela UART0.

Periféricos utilizados

SYSCLK
Watchdog Timer
Crossbar
GPIO
UART0
Timer1
Timer0
PCA0
Sistema de interrupções, UART0 e PCA0

Configuração dos periféricos utilizados

SYSCLK e Watchdog

A função de inicialização começa por desativar o Watchdog Timer, evitando reinícios automáticos durante a execução do programa. De seguida, é seleccionado o oscilador interno de alta frequência como fonte de clock do sistema. O registo PFE0CN também é configurado para garantir o correto funcionamento do acesso à memória flash a frequências elevadas.

```
WDTCN = 0xDE;  
WDTCN = 0xAD;  
  
CLKSEL = 0x00;  
PFE0CN |= (1 << 4);  
CLKSEL |= 0x03;
```

Assim, os registos principais envolvidos nesta configuração são WDTCN, CLKSEL e PFE0CN.

CROSSBAR

A crossbar é usada para encaminhar os sinais dos periféricos para os pinos físicos do microcontrolador. Neste projeto, a UART0 é encaminhada pela crossbar e o sinal CEX0 do PCA é disponibilizado para a saída PWM. A crossbar é ativada globalmente no registo XBR2.

```
XBR0 = 0x01; // ativa UART0
XBR1 = 0x01; // ativa PCA/CEX0
XBR2 = 0x40; // ativa globalmente a crossbar
```

Os registos P0SKIP, P1SKIP e P2SKIP são usados para indicar à crossbar quais os pinos que devem ser ignorados. Desta forma, P0.4 e P0.5 ficam reservados para a UART0 e P2.0 fica disponível para o sinal PWM CEX0.

```
P0SKIP = 0xCF;
P1SKIP = 0xFF;
P2SKIP = 0xFE;
```

GPIO

Os pinos usados pelos periféricos são configurados como saídas push-pull. O pino P0.4 é usado como saída de transmissão da UART0. O pino P2.0 é usado como saída do sinal PWM gerado pelo PCA0, através do canal CEX0.

```
P0MDOUT |= (1 << 4); // TX da UART0 em push-pull
P2MDOUT |= (1 << 0); // saída PWM CEX0 em push-pull
```

Quando a saída PWM está desligada, o programa força o pino P2.0 para o estado desligado através do porto P2.

```
P2 |= 0x01;
```

UART0 e Timer1

A UART0 é usada em modo série para receber comandos ASCII vindos do terminal e transmitir respostas. A comunicação é configurada para 115200 bps. O baudrate é gerado pelo Timer1 em modo 2, isto é, modo de 8 bits com auto-reload.

```
CKCON0 |= 0x08;

TCON_TR1 = 0;
TMOD &= ~0xF0;
TMOD |= 0x20;

TH1 = 43;
TL1 = 43;

TCON_TR1 = 1;
```

A UART0 é configurada através do registo SCON0. O valor 0x50 configura a UART em modo série de 8 bits e ativa a receção. As flags de transmissão e receção são limpas antes de iniciar o funcionamento.

```
SCON0 = 0x50;
SCON0_TI = 0;
SCON0_RI = 0;
IE_ES0 = 1;
```

A receção é feita por interrupção. Quando um carácter chega à UART0, a rotina de interrupção lê o valor recebido em SBUF0 e coloca-o na FIFO de receção. A transmissão também usa uma FIFO, enviando os caracteres pela UART quando o transmissor se encontra livre.

As respostas transmitidas pelo programa podem ser OK, ERR ou tramas de estado no formato da Fase 2. Um exemplo de resposta de estado é:

```
T07OFFD100*6B
```

Timer0

O Timer0 é configurado em modo 2, ou seja, modo de 8 bits com auto-reload. O clock do Timer0 é configurado através do registo CKCON0, usando SYSCLK/48. O overflow do Timer0 é usado como fonte temporal do PCA0.

```
TCON_TR0 = 0;

TMOD &= ~0x0F;
TMOD |= 0x02;

CKCON0 &= ~0x07;
CKCON0 |= 0x02;
```

O valor de recarga é calculado em função da frequência PWM pretendida. Como o PCA está configurado em PWM de 8 bits, cada período PWM corresponde a 256 contagens do PCA. O programa calcula o valor de reload necessário para obter frequências entre 100 Hz e 1000 Hz.

```
timer0_clk = 49000000UL / 48UL;
wanted_pca_clk = ((unsigned long)freq) * 256UL;
counts = (timer0_clk + (wanted_pca_clk / 2UL)) / wanted_pca_clk;
reload = (unsigned char)(256UL - counts);

TH0 = reload;
TL0 = reload;
```

Depois da configuração, a flag de overflow é limpa e o Timer0 é arrancado.

```
TCON_TF0 = 0;
TCON_TR0 = 1;
```

PCA0

O PCA0 é configurado para usar o overflow do Timer0 como fonte de clock. O módulo 0 do PCA é configurado em modo PWM de 8 bits e o módulo 1 é usado como software timer, com interrupção, para avançar periodicamente os pontos da sequência.

```
PCA0CN0_CR = 0;

PCA0MD = 0x04; // PCA usa overflow do Timer0
PCA0CPM0 = 0x42; // módulo 0 em PWM de 8 bits
PCA0CPM1 = 0x49; // módulo 1 como temporizador com interrupção
```

O duty-cycle é aplicado escrevendo o valor calculado nos registos do canal PWM. O valor de comparação é obtido a partir do duty-cycle em percentagem e convertido para a escala de 8 bits.

```
cmp = (unsigned char)((((unsigned int)duty * 255u) / 100u);

PCA0CPL0 = cmp;
PCA0CPH0 = cmp;
```

O contador do PCA e as flags são inicializados antes de ativar a interrupção. O módulo 1 começa com um primeiro valor de comparação definido em PCA0CPL1 e PCA0CPH1.

```
PCA0L = 0x00;
PCA0H = 0x00;

PCA0CPL1 = 0x00;
PCA0CPH1 = 0x01;

PCA0CN0_CF = 0;
PCA0CN0_CCF1 = 0;

EIE1 |= 0x10;
```

Durante a reprodução da sequência, o módulo 1 do PCA gera interrupções periódicas. Em cada interrupção é atualizado o índice da sequência e aplicado um novo duty-cycle ao módulo PWM. O contador auxiliar seq_delay reduz a velocidade de atualização, tornando a variação visível.

Comando STOP e saída PWM

O comando STOP não desliga todo o contador do PCA. Em vez disso, desativa temporariamente o módulo PWM e força o pino da saída para o estado desligado. Assim, o duty-cycle atual é preservado e pode ser repostado quando o comando START voltar a ativar a saída PWM.

```
PCA0CPM0 = 0x00;
P2 |= 0x01;
pwm_enabled = 0;
```

Sistema de interrupções

O programa utiliza interrupções para a UART0 e para o PCA0. A interrupção da UART0 permite receber e transmitir caracteres sem bloquear o ciclo principal. A interrupção do PCA0 permite atualizar automaticamente a sequência de duty-cycles.

```
IE_ES0 = 1;    // ativa interrupção da UART0
EIE1 |= 0x10;  // ativa interrupção do PCA0
IE_EA = 1;     // ativa interrupções globais
```

A rotina de interrupção da UART0 usa o vetor interrupt 4. A rotina de interrupção do PCA0 usa o vetor interrupt 11. No ciclo principal, a função `protocol_task()` retira os caracteres recebidos da FIFO da UART0 e processa-os pela máquina de estados do protocolo.

Resumo dos periféricos e registos principais

Periférico	Função no projeto	Registos principais
SYSCLK / Watchdog	Configuração do clock do sistema e desativação do watchdog.	WDTCN, CLKSEL, PFE0CN
Crossbar	Encaminhamento da UART0 e do sinal CEX0 para os pinos físicos.	XBR0, XBR1, XBR2, P0SKIP, P1SKIP, P2SKIP
GPIO	Configuração dos pinos usados como saídas push-pull.	P0MDOUT, P2MDOUT, P2
UART0	Comunicação série com o terminal.	SCON0, SBUF0, IE_ES0
Timer1	Geração do baudrate da UART0.	CKCON0, TMOD, TH1, TL1, TCON_TR1
Timer0	Base temporal do PCA0/PWM.	CKCON0, TMOD, TH0, TL0, TCON_TR0, TCON_TF0
PCA0	Geração do PWM e temporização da sequência.	PCA0MD, PCA0CPM0, PCA0CPM1, PCA0CPL0, PCA0CPH0, PCA0CN0
Interrupções	Tratamento assíncrono da UART0 e do PCA0.	IE_ES0, EIE1, IE_EA

Algoritmo geral em Pseudocódigo

INÍCIO

- Inicializar microcontrolador
 - Desativar watchdog
 - Configurar clock do sistema
 - Configurar GPIO
 - Configurar crossbar
 - Configurar Timer1 para UART0
 - Inicializar UART0 com interrupções

Inicializar FIFOs de transmissão e receção

Inicializar PWM

Configurar Timer0 como clock do PCA

Definir frequência inicial do PWM = 500 Hz

Configurar PCA em modo PWM de 8 bits

Definir duty-cycle inicial = 50%

Manter saída PWM desligada inicialmente

Inicializar variáveis da sequência

Ativar interrupções globais

ENQUANTO verdadeiro FAZER

Executar tarefa do protocolo

FIM ENQUANTO

FIM

Diagramas de Estados

A receção e processamento das tramas pode ser representada através de uma máquina de estados (FSM). A Figura 1 apresenta os estados utilizados na receção de caracteres pela UART e as respetivas transições entre estados.

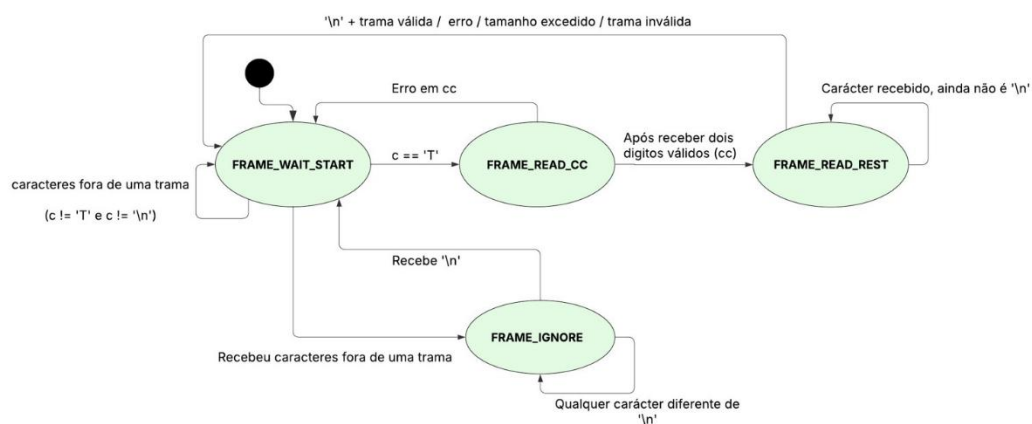


Figura 1- FSM receção da trama

O protocolo implementado possui ainda uma FSM principal responsável por distinguir o modo normal de processamento de comandos do modo de carregamento de sequências. A Figura 2 apresenta os estados e transições associados a esta funcionalidade.

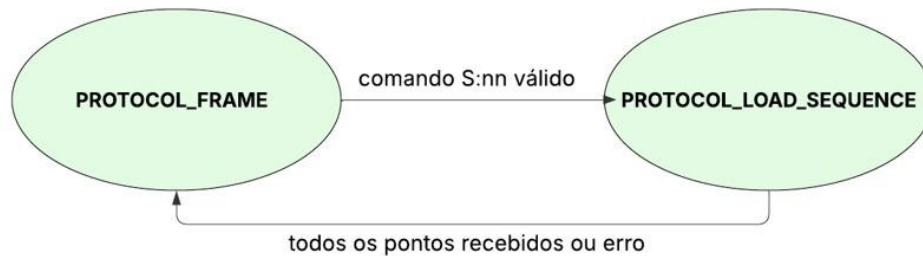


Figura 2 - FSM principal do protocolo

Programa final em linguagem C

O programa final encontra-se dividido em módulos, permitindo separar a inicialização, a comunicação série, o processamento do protocolo, a execução dos comandos e o controle do PWM. Para a entrega, o código final foi organizado nos seguintes módulos:

Módulo	Função
main.c	Inicialização geral e ciclo principal
init.c / init.h	Configuração inicial do microcontrolador e pinos
uart0.c / uart0.h	Driver da UART0, recepção por interrupção e transmissão de caracteres/string
pdata_fifo.c / pdata_fifo.h	FIFO usado pelo driver da UART para armazenar caracteres recebidos
protocol.c / protocol.h	Máquina de estados de recepção, validação de tramas, checksum e modo de sequência
commands.c / commands.h	Interpretação dos comandos e envio das respostas OK, ERR e estado
pwm.c / pwm.h	Configuração do PCA0/PWM, duty-cycle, frequência, START/STOP e reprodução da sequência

Extensão da Fase2 com checksum

Na Fase 2, o protocolo de comunicação foi estendido através da introdução de um campo de checksum, com o objetivo de aumentar a fiabilidade da comunicação série. Esta extensão permite ao microcontrolador verificar se a trama recebida pela UART foi transmitida corretamente ou se ocorreu algum erro durante a comunicação.

Com esta alteração, o programa passou a aceitar tramas no seguinte formato:

*Tcc<dados>*hh\n*

Neste formato, o carácter T indica o início da trama, cc representa o número de caracteres existentes no campo de dados, <dados> contém o comando a executar, * funciona como separador, hh corresponde ao checksum em hexadecimal ASCII e \n indica o fim da trama.

O método adotado para o cálculo do checksum consiste na soma dos códigos ASCII dos caracteres pertencentes à parte principal da trama, ou seja, desde o carácter inicial T até ao fim do campo de dados. Assim, o checksum é calculado sobre:

Tcc<dados>

O separador *, os dois dígitos hexadecimais do checksum e o terminador \n não entram no cálculo.

O cálculo pode ser representado pela seguinte expressão:

checksum = soma ASCII dos caracteres *Tcc<dados>*, módulo 256

No programa, este cálculo é realizado acumulando os códigos ASCII dos caracteres numa variável do tipo unsigned char:

```
cs = 0;
```

```
for (i = 0; i < len; i++)  
{  
    cs = cs + (unsigned char)ff[i];  
}
```

Como a variável cs é do tipo unsigned char, o resultado fica automaticamente limitado a 8 bits. Desta forma, apenas os 8 bits menos significativos da soma são considerados, o que corresponde a uma operação módulo 256.

Após receber uma trama, o programa calcula novamente o checksum com base nos caracteres recebidos e compara esse valor com o checksum enviado no campo hh. Se ambos os valores coincidirem, a trama é considerada válida e o comando é executado. Caso contrário, a trama é rejeitada e é enviada uma resposta de erro.

Como exemplo, podemos considerar a trama:

*T04D100*8D\n*

Neste caso, o cálculo do checksum é feito apenas sobre os caracteres:

T04D100

A soma dos respetivos códigos ASCII é:

'T' = 0x54

'0' = 0x30

'4' = 0x34

'D' = 0x44

'1' = 0x31

'0' = 0x30

'0' = 0x30

Somando estes valores obtém-se:

Soma = 0x18D

Como o checksum é limitado a 8 bits, consideram-se apenas os 8 bits menos significativos:

Checksum = 0x8D

Assim, o checksum enviado na trama é 8D, ficando a trama completa:

*T04D100*8D\n*

Este método foi escolhido por ser simples e adequado à implementação no microcontrolador, uma vez que exige apenas operações de soma e comparação. Apesar de ser um método simples, permite detetar grande parte dos erros de transmissão, como alterações, perdas ou corrupções de caracteres na trama recebida.

Se for recebida uma trama sem checksum, com comprimento errado, com * fora da posição esperada, com caracteres hexadecimais inválidos ou com checksum diferente do calculado, o programa rejeita a trama e responde com ERR. Apenas após a validação do comprimento, posição do separador '*', formato hexadecimal do checksum e comparação entre checksum recebido e calculado é que o comando é executado.

Testes finais recomendados

Para validar a implementação final, podem ser usados os seguintes testes no terminal UART. Deve ser enviado *Enter* no fim de cada linha.

Testes de duty_cycle

Trama enviada	Resultado esperado
T04D100*8D	OK; duty-cycle guardado passa para 100%
T03D50*60	OK; duty-cycle guardado passa para 50%
T02D+*25	OK; incrementa duty-cycle em 1%
T02D-*27	OK; decrementa duty-cycle em 1%
T02D?*39	Devolve o estado atual com ON/OFF e o duty-cycle guardado

Testes START/STOP

Trama enviada	Resultado esperado
T05START*47	OK; saída PWM ativada
T04STOP*FE	OK; saída PWM desativada

Testes de Frequência

Trama enviada	Resultado esperado
T04F100*8F	OK; frequência passa para 100 Hz
T04F500*93	OK; frequência passa para 500 Hz
T05F1000*C0	OK; frequência passa para 1000 Hz

Testes da Sequência

Trama enviada	Resultado esperado
T04S:06*AB	Inicia carregamento de sequência com 6 pontos
0	Ponto 1
20	Ponto 2
50	Ponto 3
100	Ponto 4
50	Ponto 5
20	Ponto 6
T03SON*A7	Inicia a reprodução da sequência
T04SOFF*E6	Para a reprodução da sequência

Resultado esperado: o LED deve variar de apagado para brilho baixo, médio, máximo e voltar a diminuir.

Testes de erro

Trama enviada	Resultado esperado
T04D100	ERR; trama sem checksum
T04D100*5A	ERR; checksum inválido
T04F050*93	ERR; frequência fora da gama permitida

Conclusão

A implementação final cumpre a receção e interpretação de comandos por UART0, o controlo da saída PWM por PCA0, a alteração de duty-cycle e frequência, a reprodução de uma sequência de pontos e a extensão da Fase 2 com a verificação da sua integridade por checksum. Os testes realizados confirmaram o correto funcionamento das funcionalidades exigidas no Guia 2. Devido à dimensão final do programa e à limitação de espaço da versão gratuita do Keil, foi necessário utilizar uma licença do Keil para permitir a compilação completa do código. Esta necessidade surgiu com a extensão do projeto, nomeadamente com a implementação do protocolo da Fase 2, da validação por checksum e da reprodução de sequências PWM.